Motion classes across the M7 / M4 boundary

M7 · UI application

«UI»
icc::Bridge
M7 instance

TopicBridge / ServiceBridge / ActionBridge
bus + coroutine context are borrowed
transport : ITransport&

«UI»
ui::Application

bus : pnm::msg::Bus
iccContext : pnm::coro::Context
shared application stop source

«UI»
Axis

Angle | Distance

robot : Robot&
joint index

- moveRel() / moveAbs() / home() / jog()

«UI»
ui::motion::Robot

bounded planning queue + workspace

- submit(ExecutionRequest)
- run() : planning worker

«UI»
ui::motion::Client

jobs + retained action handles
history : 32 entries

- submit(MotorGoal)
- acquire() / execute() / cancel()
- run() : dispatch + lease renewals

«UI»
ui::motion::Planner

RobotModel + workspace references

- prepare(move, start)
- finite() : FiniteProgram

Serialized frames only
SRAM4 + HSEM

local bus endpoints
(no direct Bridge call)

forwards submission

borrows; jobs survive planning

creates on planning worker

M4 · IO application

«IO»
icc::Bridge
M4 instance

TopicBridge / ServiceBridge / ActionBridge
bus + coroutine context are borrowed
transport : ITransport&

«IO»
io::Application

bus : pnm::msg::Bus
iccContext : pnm::coro::Context
shared application stop source

dispatch via private bus

«IO»
io::motion::Server

deferred services + action providers
command worker + completion records

- run() / admitProgram() / dispatchPrograms()

borrowed pointer; authoritative commands

«IO»
io::motion::Controller

one execution worker
groups[3] + driver monitoring

- control() / prepareProgram() / startProgram()
- moveRel() / moveAbs() / reference()

1

1

owned by groups
0..3

«IO»
Ownership

family + axis masks + job IDs
reservation tokens + revisions + lease deadlines

owns shared handle / refill worker

«IO»
Executor

finite / jog program execution
shared progress and installed stopping branch

- prepare() / bind() / start() / service()
- renew() / stop() / abort()

shared backend

immutable schedule banks

prepare callback
sets DIR and binds output

3

hal::IStepGenerator

shared timer / DMA backend
per-axis IStepOutput handles

- preparePending() / splice()

«IO»
ProfileSchedule

TimedSequence implementation
nominal profile + optional braking suffix

- generate() : pulse timings

«IO»
StepperMotor

raw moves + buffered successor + homing
reserved reusable stack per motor
stepOutput : IStepOutput
driver : IStepperDriver
feedback : IMotorFeedback
reference input + DIR output

UML relation	Meaning
filled diamond	owns / controls lifetime
solid arrow	local reference or interaction; bus mediation is labelled
dotted arrow	dependency or serialized wire interaction, as labelled

No pointer, callback, future or C++ object is shared across the core boundary.
Both builds compile the same contracts; each application owns its private objects.
Axis forwards to Robot. Raw/homing use StepperMotor; finite axis/robot programs use Executor.
Robot jobs reserve all axes. Motor / Axis / Robot command families exclude each other.
Each Bridge uses a local InterCoreTransport; no cross-core method calls occur.
Contracts: common/icc/catalog/{services,actions,topics}.hpp
Normal: io/trajectory, io/motion, io/motor/motion. Urgent: io/trajectory/control, io/motor/stop.
State/log topics: io/motor/state, io/controller/state, io/logs.

Selected classes and responsibilities; separate instances and address spaces on each core.

2026-11-06 | Current source structure; hardware qualification remains incomplete.

Axis / robot finite move · reservation, planning and start

M7 · UI

M4 · IO

CLI caller

Robot planning worker

Client job registry

Serialized ICC normal lane

Server dispatch + command worker

Controller + Ownership

Executor

STEP HAL TIM2 / DMA

1 submit(ExecutionRequest) Axis facade or Robot move

2 create(family, selected axes)

3 job ID

4 enqueue planning request; notify

5 job ID immediately

Returning to the prompt does not destroy the job.
Robot.run() performs planning on its dedicated worker.

Acquire an authoritative starting state

6 inspect() / model validation

7 io/trajectory: Inspect

8 request ready; notify IO dispatcher

9 control(Inspect)

10 boot ID, capabilities, positions, references

11 Reply

12 decoded reply

13 model / hardware capabilities

14 acquire(job, configuration fingerprint, axes)

15 io/trajectory: Acquire

16 dispatch deferred service

17 control(Acquire)

18 atomic family / axis admission

alt **[Busy, stale boot, unavailable hardware or invalid configuration]**

19 error + busy job when applicable

20 rejection

21 mark Rejected

22 stop planning this request

[Reservation obtained]

23 token + commanded positions + axis revisions

24 authoritative reservation snapshot

25 store boot / token with job

26 snapshot

Client.run() renews active ownership every 50 ms.
M4 expires an unrenewed lease after 250 ms.

27 convert units, validate path, quantize endpoint
compile one normalized progress profile

FiniteProgram contains profile description,
signed displacements and axis limits.
Cartesian point moves interpolate in joint space.

Prepare locally on M4; outputs are not started

28 execute(job, reservation, FiniteProgram)

29 io/trajectory: Prepare boot + token + configuration + positions + revisions

30 deferProgram(): queue bounded command

31 [command worker] prepareProgram()

32 verify lease, revisions, positions, references, limits

33 prepare(command, bind_outputs=false)

Numerical preparation runs outside Controller's mutex.
Reservation keeps the selected axes owned.

34 validate progress; build nominal + stopping schedules

35 prepared or error

36 reacquire mutex; revalidate epoch / reservation / axes

alt **[Preparation fails, is cancelled, or becomes stale]**

37 explicit rejection; no start

38 deferred Reply

39 reject job; request reservation Release

[Preparation remains valid]

40 bind() via StepperMotor::prepareCoordinated()

41 bind per-axis schedules / DIR; stage initial buffers

42 Staged

43 finishProgramCommand(): deferred Reply

44 preparation succeeded

Run is a separate action

45 io/motion goal: Run{boot, token, job}

46 admitProgram(Run, execution)

47 startProgram(Run, completion callback)

48 final ownership / revision / hardware checks

49 start(token)

50 start prepared outputs on common timeline

51 success

52 action accept()

53 on_accepted → Running

In the current code, startProgram() precedes action acceptance.
A STEP edge may occur before the UI receives acceptance.
Zero-displacement programs complete without arming outputs.

Source-level successful path plus rejection gates. This is not a hardware qualification claim.

ICC lifeline = both bridges + transports + queues; it is not an additional application class.

Finite motion · execution, controlled stop and lease loss

M7 · UI

Client application worker

Serialized ICC urgent / normal lanes

M4 · IO

Server IO dispatcher

Controller execution worker

Executor

STEP HAL TIM2 / DMA

Two different renewals

loop [UI owner keepalive · every 50 ms while active]

1 urgent io/trajectory/control: Renew{job IDs}

2 urgent service ready

3 control(Renew) under Controller mutex

4 renew valid owner deadlines to now + 250 ms

5 acknowledgement

6 urgent reply

7 record acknowledgement / error

One renewal request in flight; late renewals do not revive expired jobs.
Ownership renewal is separate from M4 pulse-schedule maintenance.

loop [M4 execution worker · notifications + up to 1 ms timed wait when active]

8 preparePending(): fill requested DMA buffers

9 check faults, ownership lease and motion timeout

10 renew(id); service()

opt [Maintenance due · nominally every 40 ms]

11 build next immutable schedule bank include stopping continuation from its boundary

12 splice(replacement, current boundary)

13 accepted, retry or error

14 advance committed frontier toward now + 80 ms

15 hardware-timed STEP edges

16 refill / completion notifications

The installed schedule already contains its brake.
Continuing motion replaces future data before that boundary.
Stopping does not need a new M7 plan.

Controlled stop · two ways to select the installed brake

alt [Explicit axis/robot stop or action cancellation]

17 urgent Stop{job or axis mask} (cancel also requests action cancellation)

18 control request / cancel notification

19 control(Stop); expand target to whole owned group

20 stop(id)

21 state = Braking; cease nominal schedule extension

22 stop request processed

23 acknowledgement

24 request acknowledged, NOT yet terminal

[UI renewal stops / link is lost]

no valid owner renewal arrives

25 owner deadline expires (or connectionLost() invalidates admission)

26 stop(id)

27 use already installed stopping continuation

No reply from M7 is needed for this local selection.
A disconnected UI cannot infer a confirmed physical stop.

Completion and ownership release

28 finish committed prefix, brake, then last STEP

29 completion notification

30 service() observes all participating outputs complete

31 Stopped + counts / error status

32 bounded completion callback: copy result + notify

33 dispatchPrograms(): observe completion record

34 terminal action result: cancelled(Stopped)

35 releaseJob(job) after completion cleanup

36 ready notification; Client.run() dispatches result

37 retain history; release pending action handle

Natural completion follows the same result path with Completed.
Communication failure can leave the UI result Unknown.
Local faults / explicit Abort bypass braking and may invalidate references.
Hardware-guard cessation is not a controlled-stop guarantee.

Starts with a prepared, running Executor program. Raw motor stops and homing use a different path.

Configured timing is shown, not a measured worst-case guarantee. Source-level behavior still needs full hardware qualification.

Raw motor command · the StepperMotor worker path

M7 · UI

M4 · IO

CLI

Client
retained job / action

Serialized ICC

Server
IO dispatcher

Controller
+ Ownership

StepperMotor
selected motor

STEP HAL
shared TIM2 / DMA

1 submit(MotorGoal)
shaft angle / rpm / dynamics / BufferMode

2 allocate job ID; retain PendingGoal

3 normal io/motor/motion goal

4 job ID; prompt returns

5 admit(goal, ActionExecution)

6 validate goal / command generation / readiness

7 reserveMotor(job, motor)

8 reject Axis / Robot ownership conflicts

alt **[Rejected or capacity exhausted]**

9 Busy owner / rejection

10 rejected action

11 terminal rejection

[Motor family admitted]

12 ownership granted

13 action accept()

14 accepted callback

15 moveRel() / moveAbs()
completion observer

16 submit to selected motor

alt **[New active move]**

create motion worker
17 reuse reserved stack
10 KiB ARM / 16 KiB Linux; priority 16

[Allowed buffered / blended successor]

18 apply existing bounded queue rules

19 future<Result>

20 future<Result> (local M4 object only)

Raw action acceptance precedes launching the motor worker.
A later launch failure produces Faulted, not a successful move.

21 runCommands() / CommandRunner
build Profile + Sequence

22 configure / start selected IStepOutput

23 compute initial edges in bounded batches
interrupts enabled; existing DMA keeps running

24 bounded publication
recheck revision / cancellation; establish arm origin

Telemetry reads generator-synchronized state without taking the motor command mutex. Startup cannot block ICC through that mutex. Scratch growth retires the old workspace before allocating its replacement; active DMA buffers are separate and remain owned.

loop **[Until completion, stop or fault]**

25 generate hardware STEP edges

26 progress / completion notification

27 update accounting; wait for notification

28 action feedback + latest motor-state topic

29 dispatch progress / state

30 stop active output; complete(current command)

31 completion observer sets ready flag + wakes server

par **[Motor worker continues]**

32 take next buffered command or finishQueue() and return

[Server dispatches the completed job]

33 get ready future; determine result

34 succeed / cancelled / abort

35 releaseJob(job)

36 terminal callback; keep history

Current raw completion reports a command result before the worker necessarily returns. The next submission joins it before stack reuse. The sequential robot-homing path explicitly calls stopAndWait().

Other raw motors may run independently; same-motor buffering retains ownership.
Raw motor stop is immediate cessation, not the Executor's coordinated brake.
A raw stop targeting a robot-owned member aborts its whole group.

Existing motor commands bypass Robot/Planner and the coordinated Executor program path.

This diagram describes current source behavior. Action acceptance and successful execution are distinct events.

Sequential robot homing · lifetime, deadlines and cleanup

M7 · UI

Client / Robot

Serialized ICC

M4 · IO

Server

Controller execution worker

StepperMotor selected stage

Runtime ThreadX

1 Acquire Robot ownership; Prepare Home; Run

2 deferred preparation / action

3 validate version, reservation, revisions, configuration

4 store order and per-axis limits
retain whole-robot ownership

5 startProgram(completion callback)

6 arm overall timeout if supplied

7 accepted

8 Running

loop **[Configured homing order, while no cancellation / fault / timeout]**

9 reference(seek, latch, stage timeout)

10 join previous worker before reusing its stack

11 create_jthread with reserved stack
10 KiB ARM / 16 KiB Linux, priority 16

Stack storage is reserved once per motor.
Thread control / TLS still need bounded runtime resources.
Creation failures are logged with motor and job identity.

12 future<Result>; attach completion notification

13 seek contact → release → slow re-latch

opt **[Overall timeout / owner lease expiry / cancellation]**

14 requestStop() for participating motors

record specific terminal reason
15 wait for active-stage completion; do not start next stage

alt **[Successful latch]**

16 establish configured reference position

[Stop / stage timeout / local fault]

17 stop output; retain unsuccessful reference state

18 ready flag + notification

19 get completed result; stopAndWait()

20 worker joined and cleanup complete

21 advance only after successful stage

22 bounded completion record + notification

23 terminal Completed / Stopped / Faulted

24 releaseJob() after cleanup

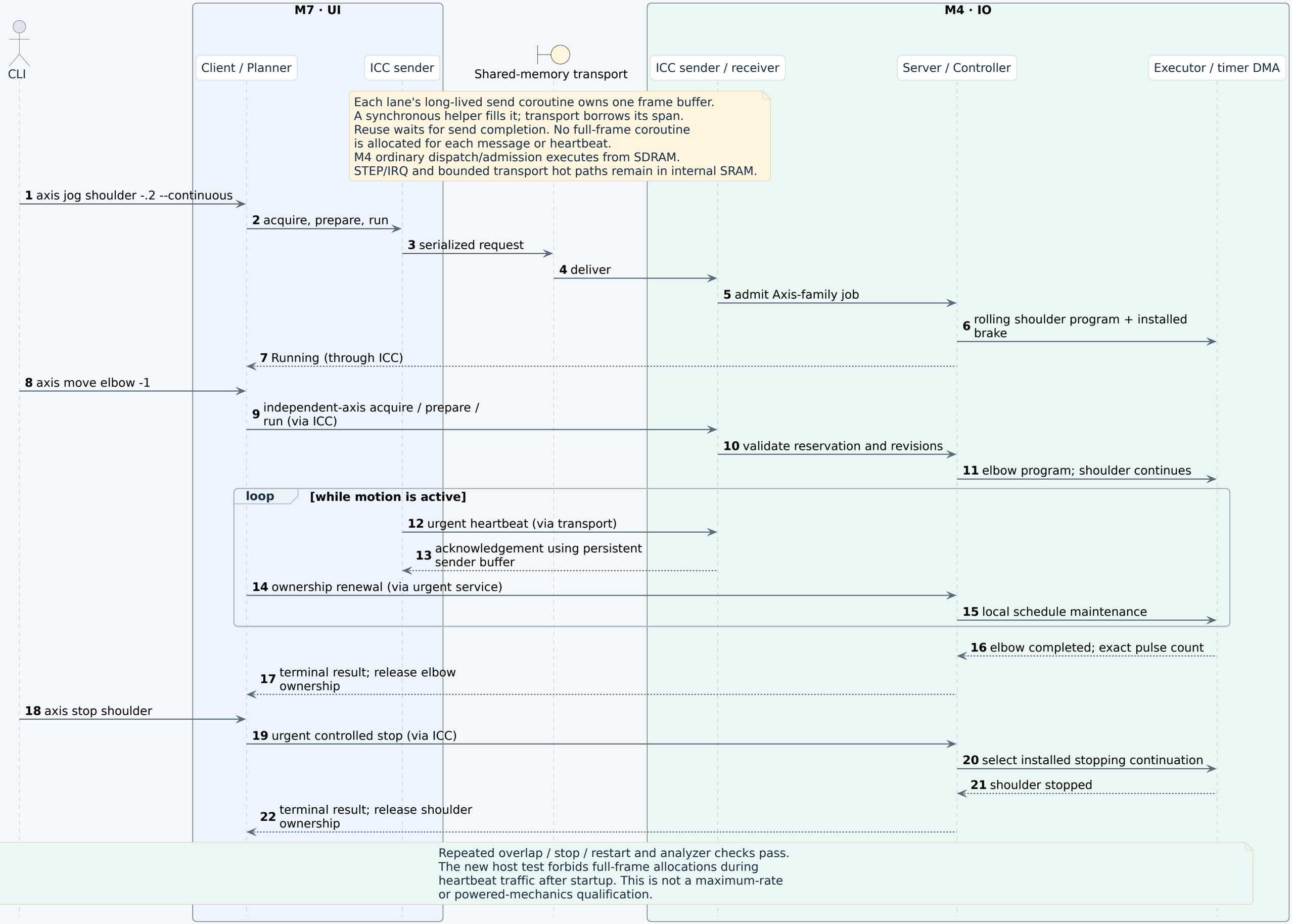
25 retained job history; notify explicit waiters

Interrupting motion wait does not cancel motion.
A stop or timeout never resumes automatically.
Only completed homing stages establish references.

The same StepperMotor seek/release/re-latch path is used for individual axis homing.

Software behavior; disconnected-board fixtures use synthetic reference switches.

Concurrent motion · independent jobs and persistent ICC buffers



Motor-disconnected application fixture; physical commissioning remains separate.

Heartbeat 100 ms / deadline 500 ms and ownership lease 250 ms are unchanged.

Native simulation · one clock, completed interrupt handoff

Native simulator process

Controller process

Session / Machine

Shared-memory
IRQ sequence + idle epoch

Interrupt thread

ThreadX scheduler

Homing worker

GUI / other workers

1 advance to next event only when prior work is idle

2 publish reference input, IRQ flags and sequence N

3 eventfd wake

4 reference callback latches activation

5 notify homing worker

6 acknowledge IRQ N

IRQ acknowledgement does not authorize more virtual time.
The notified homing worker must get a chance to run.

7 leave interrupt context

8 dispatch ready worker in priority order

9 HAL transaction: stop STEP; hold virtual time

10 process register requests

11 output stopped, exact count available

12 wait for debounce / next event

13 dispatch remaining runnable work

14 suspend until notification / simulated deadline

15 no runnable threads, no ISR: idle_epoch = N + 1

16 matching IRQ acknowledgement and idle epoch

17 next event may advance virtual time

ThreadX ticks, STEP, feedback and display VSYNC share virtual time.
SDL input pumping and native peer watchdogs remain on wall time.
Slow host computation stretches wall duration instead of starving a worker behind repeated simulated timer interrupts. GIGA firmware is unchanged.

Linux functional verification; this does not model Cortex-M execution time.

Source: Session.cpp; simulation/Client.cpp; ThreadX Linux scheduler hook; DisplayHost.cpp.